Computer Science and Engineering Department, SVNIT, Surat
M.Tech. I – Semester I
High Performance Computing (CSDS125)
Lab Assignment 7: Task Allocation using CUDA Genetic Algorithm and Particle Swarm Optimization

Objective

To implement and compare two population-based optimization algorithms—Genetic Algorithm (GA) and Particle Swarm Optimization (PSO)—for assigning tasks to processors. CUDA is used to evaluate candidate task allocations in parallel on the GPU.

Problem Description

There are 200 independent tasks and 8 processors. Each task has a positive execution time. The goal is to assign every task to one processor while minimizing the *makespan*, defined as the maximum total execution time assigned to any processor.

If t_i is the execution time of task i and a_i is its assigned processor, the load of processor j is

$$L_j = \sum_{i:\, a_i = j} t_i,$$

and the objective is

$$\min \; C_{\max} = \min \left(\max_{j=0,\ldots,7} L_j \right).$$

Both programs initialize task execution times to integers between 10 and 100 using a fixed seed for reproducibility. The configurations are 200 tasks, 8 processors, and 512 candidate solutions. GA runs for 500 generations; PSO runs for 500 iterations.

Common Experimental Configuration

Parameter	Value
Number of tasks	200
Number of processors	8
GA population size / PSO particles	512
GA generations / PSO iterations	500
Task execution-time range	10–100 time units
Random seed	42
Objective	Minimize makespan

Algorithm 1: Genetic Algorithm (GA)

Each chromosome contains one gene per task; the gene stores the processor ID assigned to that task. Fitness is the makespan, so a lower fitness value is better. The algorithm uses tournament selection, single-point crossover, random mutation, and elitism. The CUDA kernel evaluates the fitness of multiple chromosomes in parallel, while the CPU performs the genetic operations.

CUDA C Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>
#include <cuda_runtime.h>

#define NUM_TASKS 200
#define NUM_PROCESSORS 8
#define POP_SIZE 512
#define GENERATIONS 500

#define ELITE_COUNT 2
#define TOURNAMENT_SIZE 3
#define MUTATION_RATE 0.03f

#define CUDA_CHECK(call) do { \
    cudaError_t err = (call); \
    if (err != cudaSuccess) { \
        fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__, cudaGetErrorString(err)); \
        exit(EXIT_FAILURE); \
    } \
} while (0)

/*
 * A chromosome is an integer array of length NUM_TASKS.
 * chromosome[t] identifies the processor assigned to task t.
 *
 * Fitness = makespan = maximum total execution time across processors.
 * Lower fitness is better.
 */

__global__ void evaluateFitness(
    const int *task_times,
    const int *population,
    int *fitness,
    int population_size,
    int num_tasks)
{
    int chromosome = blockIdx.x * blockDim.x + threadIdx.x;
    if (chromosome >= population_size) return;

    int loads[NUM_PROCESSORS];
    for (int p = 0; p < NUM_PROCESSORS; ++p) {
        loads[p] = 0;
    }

    for (int t = 0; t < num_tasks; ++t) {
        int processor = population[chromosome * num_tasks + t];

        if (processor < 0) processor = 0;
        if (processor >= NUM_PROCESSORS) processor = NUM_PROCESSORS - 1;

        loads[processor] += task_times[t];
    }

    int makespan = 0;
    for (int p = 0; p < NUM_PROCESSORS; ++p) {
        if (loads[p] > makespan) {
            makespan = loads[p];
        }
    }

    fitness[chromosome] = makespan;
}

```

```
static int tournamentSelection(const int *fitness)
{
    int best = rand() % POP_SIZE;

    for (int i = 1; i < TOURNAMENT_SIZE; ++i) {
        int candidate = rand() % POP_SIZE;
        if (fitness[candidate] < fitness[best]) {
            best = candidate;
        }
    }

    return best;
}

static void copyChromosome(
    const int *source,
    int source_index,
    int *destination,
    int destination_index)
{
    for (int t = 0; t < NUM_TASKS; ++t) {
        destination[destination_index * NUM_TASKS + t] =
            source[source_index * NUM_TASKS + t];
    }
}

static void sortIndicesByFitness(const int *fitness, int *indices)
{
    for (int i = 0; i < POP_SIZE; ++i) {
        indices[i] = i;
    }

    // Population is modest in size; selection sort keeps this example simple.
    for (int i = 0; i < POP_SIZE - 1; ++i) {
        int best_pos = i;
        for (int j = i + 1; j < POP_SIZE; ++j) {
            if (fitness[indices[j]] < fitness[indices[best_pos]]) {
                best_pos = j;
            }
        }

        if (best_pos != i) {
            int tmp = indices[i];
            indices[i] = indices[best_pos];
            indices[best_pos] = tmp;
        }
    }
}

int main(void)
{
    srand(42);

    int task_times[NUM_TASKS];

    // Initialize every task with a nonzero execution time before GPU transfer.
    for (int t = 0; t < NUM_TASKS; ++t) {
        task_times[t] = 10 + rand() % 91;  // inclusive range: 10 to 100
    }

    printf("=== Task Allocation using CUDA Genetic Algorithm ===\n");
    printf("Tasks: %d | Processors: %d | Population: %d | Generations: %d\n\n",
           NUM_TASKS, NUM_PROCESSORS, POP_SIZE, GENERATIONS);

    printf("Task execution times:\n");
    for (int t = 0; t < NUM_TASKS; ++t) {
        printf("T%-3d: %3d%s", t, task_times[t],
               ((t + 1) % 8 == 0 || t == NUM_TASKS - 1) ? "\n" : "  ");
    }

    const size_t task_bytes = NUM_TASKS * sizeof(int);
    const size_t population_bytes = (size_t)POP_SIZE * NUM_TASKS * sizeof(int);
    const size_t fitness_bytes = (size_t)POP_SIZE * sizeof(int);

    int *d_task_times = NULL;
    int *d_population = NULL;
    int *d_fitness = NULL;

    CUDA_CHECK(cudaMalloc(&d_task_times, task_bytes));
    CUDA_CHECK(cudaMalloc(&d_population, population_bytes));
    CUDA_CHECK(cudaMalloc(&d_fitness, fitness_bytes));

```

```
    CUDA_CHECK(cudaMemcpy(d_task_times, task_times, task_bytes, cudaMemcpyHostToDevice));

    int *population = (int *)malloc(population_bytes);
    int *new_population = (int *)malloc(population_bytes);
    int *fitness = (int *)malloc(fitness_bytes);
    int *indices = (int *)malloc(POP_SIZE * sizeof(int));
    int *best_chromosome = (int *)malloc(NUM_TASKS * sizeof(int));

    if (!population || !new_population || !fitness || !indices || !best_chromosome) {
        fprintf(stderr, "Host memory allocation failed.\n");
        return EXIT_FAILURE;
    }

    // Randomly initialize the population: every gene is a valid processor ID.
    for (int i = 0; i < POP_SIZE * NUM_TASKS; ++i) {
        population[i] = rand() % NUM_PROCESSORS;
    }

    const int threads = 128;
    const int blocks = (POP_SIZE + threads - 1) / threads;

    // Evaluate initial population.
    CUDA_CHECK(cudaMemcpy(d_population, population, population_bytes, cudaMemcpyHostToDevice));
    evaluateFitness<<<blocks, threads>>>(d_task_times, d_population, d_fitness,
                                         POP_SIZE, NUM_TASKS);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaMemcpy(fitness, d_fitness, fitness_bytes, cudaMemcpyDeviceToHost));

    int best_fitness = fitness[0];
    int best_index = 0;
    for (int i = 1; i < POP_SIZE; ++i) {
        if (fitness[i] < best_fitness) {
            best_fitness = fitness[i];
            best_index = i;
        }
    }
    copyChromosome(population, best_index, best_chromosome, 0);

    printf("\nInitial best makespan: %d\n", best_fitness);

    for (int generation = 1; generation <= GENERATIONS; ++generation) {
        sortIndicesByFitness(fitness, indices);

        // Elitism: copy the best ELITE_COUNT chromosomes unchanged.
        for (int e = 0; e < ELITE_COUNT; ++e) {
            copyChromosome(population, indices[e], new_population, e);
        }

        int next = ELITE_COUNT;

        while (next < POP_SIZE) {
            int parent1 = tournamentSelection(fitness);
            int parent2 = tournamentSelection(fitness);

            int crossover_point = 1 + rand() % (NUM_TASKS - 1);

            // Single-point crossover creates one or two children.
            for (int t = 0; t < NUM_TASKS; ++t) {
                int child1_gene = (t < crossover_point)
                    ? population[parent1 * NUM_TASKS + t]
                    : population[parent2 * NUM_TASKS + t];

                int child2_gene = (t < crossover_point)
                    ? population[parent2 * NUM_TASKS + t]
                    : population[parent1 * NUM_TASKS + t];

                if (((float)rand() / (float)RAND_MAX) < MUTATION_RATE) {
                    child1_gene = rand() % NUM_PROCESSORS;
                }
                if (((float)rand() / (float)RAND_MAX) < MUTATION_RATE) {
                    child2_gene = rand() % NUM_PROCESSORS;
                }

                new_population[next * NUM_TASKS + t] = child1_gene;

                if (next + 1 < POP_SIZE) {
                    new_population[(next + 1) * NUM_TASKS + t] = child2_gene;
                }
            }

            next += 2;
```

```
        }

        // Replace old population with offspring.
        int *swap = population;
        population = new_population;
        new_population = swap;

        // Evaluate new generation on GPU.
        CUDA_CHECK(cudaMemcpy(d_population, population, population_bytes, cudaMemcpyHostToDevice));
        evaluateFitness<<<blocks, threads>>>(d_task_times, d_population, d_fitness,
                                             POP_SIZE, NUM_TASKS);
        CUDA_CHECK(cudaGetLastError());
        CUDA_CHECK(cudaDeviceSynchronize());
        CUDA_CHECK(cudaMemcpy(fitness, d_fitness, fitness_bytes, cudaMemcpyDeviceToHost));

        // Update global best-so-far. It can only stay the same or improve.
        int generation_best = fitness[0];
        int generation_best_index = 0;
        for (int i = 1; i < POP_SIZE; ++i) {
            if (fitness[i] < generation_best) {
                generation_best = fitness[i];
                generation_best_index = i;
            }
        }

        if (generation_best < best_fitness) {
            best_fitness = generation_best;
            copyChromosome(population, generation_best_index, best_chromosome, 0);
        }

        if (generation == 1 || generation % 25 == 0 || generation == GENERATIONS) {
            printf("Generation %3d | Current generation best: %4d | Best so far: %4d\n",
                   generation, generation_best, best_fitness);
        }
    }

    // Compute final loads from the best chromosome found over all generations.
    int processor_loads[NUM_PROCESSORS] = {0};
    int processor_task_counts[NUM_PROCESSORS] = {0};
    long long total_time = 0;

    for (int t = 0; t < NUM_TASKS; ++t) {
        int processor = best_chromosome[t];
        if (processor < 0) processor = 0;
        if (processor >= NUM_PROCESSORS) processor = NUM_PROCESSORS - 1;

        processor_loads[processor] += task_times[t];
        processor_task_counts[processor]++;
        total_time += task_times[t];
    }

    int final_makespan = 0;
    for (int p = 0; p < NUM_PROCESSORS; ++p) {
        if (processor_loads[p] > final_makespan) {
            final_makespan = processor_loads[p];
        }
    }

    printf("\n=== Final Genetic Algorithm Result ===\n");
    printf("Best makespan: %d\n", final_makespan);
    printf("Total task execution time: %lld\n", total_time);
    printf("Ideal lower bound (ceil(total time / processors)): %lld\n",
           (total_time + NUM_PROCESSORS - 1) / NUM_PROCESSORS);

    printf("\nProcessor summary:\n");
    for (int p = 0; p < NUM_PROCESSORS; ++p) {
        printf("P%d: load = %4d | tasks = %3d\n",
               p, processor_loads[p], processor_task_counts[p]);
    }

    printf("\nTask assignments:\n");
    for (int t = 0; t < NUM_TASKS; ++t) {
        printf("T%-3d (time %3d) -> P%d\n",
               t, task_times[t], best_chromosome[t]);
    }

    free(population);
    free(new_population);
    free(fitness);
    free(indices);
    free(best_chromosome);

```

```
    CUDA_CHECK(cudaFree(d_task_times));
    CUDA_CHECK(cudaFree(d_population));
    CUDA_CHECK(cudaFree(d_fitness));

    return 0;
}
```

GA Output

Compile and run the program using:

```
nvcc -O2 ga_task_allocation.cu -o ga
./ga
```

The program prints all task execution times, the best makespan at selected generations, final processor loads, and each task's assignment. The following is the output format; numerical values are produced by the actual CUDA run.

```
=== Task Allocation using CUDA Genetic Algorithm ===
Tasks: 200 | Processors: 8 | Population: 512 | Generations: 500

Task execution times:
T0  : ...  T1  : ...  ...  T199: ...

Initial best makespan: <value>
Generation   1 | Current generation best: <value> | Best so far: <value>
Generation  25 | Current generation best: <value> | Best so far: <value>
...
Generation 500 | Current generation best: <value> | Best so far: <value>

=== Final Genetic Algorithm Result ===
Best makespan: <value>
Total task execution time: <value>
Ideal lower bound (ceil(total time / processors)): <value>

Processor summary:
P0: load = <value> | tasks = <value>
...
P7: load = <value> | tasks = <value>

Task assignments:
T0 (time <value>) -> P<id>
...
T199 (time <value>) -> P<id>
```

Algorithm 2: Particle Swarm Optimization (PSO)

Each particle represents a candidate assignment. A continuous position for each task is clipped to the valid processor range and converted to a processor ID. Particle velocities and positions are updated using inertia, the particle's personal best, and the swarm's global best. The CUDA kernels update particle positions and evaluate their makespan in parallel; the CPU maintains personal-best and global-best solutions.

CUDA C Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>
```

```cpp
#include <cuda_runtime.h>

#define NUM_TASKS 200
#define NUM_PROCESSORS 8
#define NUM_PARTICLES 512
#define MAX_ITERATIONS 500

#define W 0.72f
#define C1 1.49f
#define C2 1.49f

#define CUDA_CHECK(call) do { \
    cudaError_t err = (call); \
    if (err != cudaSuccess) { \
        fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__, cudaGetErrorString(err)); \
        exit(EXIT_FAILURE); \
    } \
} while (0)

/*
 * A particle stores one continuous position per task.
 * Each position is mapped to a processor by:
 *     assignment = floor(position)
 * clipped to [0, NUM_PROCESSORS - 1].
 *
 * Fitness is the makespan: maximum total task time on any processor.
 */

__global__ void evaluateFitness(
    const int *task_times,
    const float *positions,
    float *fitness,
    int num_particles,
    int num_tasks)
{
    int particle = blockIdx.x * blockDim.x + threadIdx.x;
    if (particle >= num_particles) return;

    // One thread evaluates one complete particle.
    int loads[NUM_PROCESSORS];
    for (int p = 0; p < NUM_PROCESSORS; ++p) loads[p] = 0;

    for (int t = 0; t < num_tasks; ++t) {
        float x = positions[particle * num_tasks + t];
        int processor = (int)floorf(x);

        if (processor < 0) processor = 0;
        if (processor >= NUM_PROCESSORS) processor = NUM_PROCESSORS - 1;

        loads[processor] += task_times[t];
    }

    int makespan = 0;
    for (int p = 0; p < NUM_PROCESSORS; ++p) {
        if (loads[p] > makespan) makespan = loads[p];
    }

    fitness[particle] = (float)makespan;
}

__global__ void updateParticles(
    float *positions,
    float *velocities,
    const float *personal_best_positions,
    const float *global_best_position,
    unsigned int *rng_states,
    int num_particles,
    int num_tasks,
    int iteration)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int total = num_particles * num_tasks;
    if (idx >= total) return;

    int particle = idx / num_tasks;

    // Simple per-element deterministic pseudo-random values.
    // This avoids requiring an additional random-number library.
    unsigned int seed = rng_states[idx];
    seed = 1664525u * seed + 1013904223u;
    float r1 = (seed & 0x00FFFFFFu) / 16777216.0f;
    seed = 1664525u * seed + 1013904223u;
```

```
    float r2 = (seed & 0x00FFFFFu) / 16777216.0f;
    rng_states[idx] = seed;

    float x = positions[idx];
    float v = velocities[idx];
    float pbest = personal_best_positions[idx];
    float gbest = global_best_position[idx];

    v = W * v + C1 * r1 * (pbest - x) + C2 * r2 * (gbest - x);

    // Limit velocity to avoid large jumps.
    if (v > 2.0f) v = 2.0f;
    if (v < -2.0f) v = -2.0f;

    x += v;

    // Keep position inside the processor-index range.
    if (x < 0.0f) {
        x = 0.0f;
        v *= -0.5f;
    }
    if (x >= (float)NUM_PROCESSORS) {
        x = (float)NUM_PROCESSORS - 0.0001f;
        v *= -0.5f;
    }

    positions[idx] = x;
    velocities[idx] = v;
}

int main(void)
{
    int task_times[NUM_TASKS];

    // Initialize every task time BEFORE copying the array to the GPU.
    // Fixed seed makes the experiment reproducible.
    srand(42);
    for (int t = 0; t < NUM_TASKS; ++t) {
        task_times[t] = 10 + rand() % 91;  // execution time from 10 to 100
    }

    printf("=== Task Allocation using CUDA PSO ===\n");
    printf("Tasks: %d | Processors: %d | Particles: %d | Iterations: %d\n\n",
           NUM_TASKS, NUM_PROCESSORS, NUM_PARTICLES, MAX_ITERATIONS);

    printf("Task execution times:\n");
    for (int t = 0; t < NUM_TASKS; ++t) {
        printf("T%-3d: %3d%s", t, task_times[t], ((t + 1) % 8 == 0 || t == NUM_TASKS - 1) ? "\n" : "  ");
    }

    const size_t task_bytes = NUM_TASKS * sizeof(int);
    const size_t position_bytes = (size_t)NUM_PARTICLES * NUM_TASKS * sizeof(float);
    const size_t particle_bytes = NUM_PARTICLES * sizeof(float);
    const size_t rng_bytes = (size_t)NUM_PARTICLES * NUM_TASKS * sizeof(unsigned int);

    int *d_task_times = NULL;
    float *d_positions = NULL;
    float *d_velocities = NULL;
    float *d_personal_best_positions = NULL;
    float *d_global_best_position = NULL;
    float *d_fitness = NULL;
    unsigned int *d_rng_states = NULL;

    CUDA_CHECK(cudaMalloc(&d_task_times, task_bytes));
    CUDA_CHECK(cudaMalloc(&d_positions, position_bytes));
    CUDA_CHECK(cudaMalloc(&d_velocities, position_bytes));
    CUDA_CHECK(cudaMalloc(&d_personal_best_positions, position_bytes));
    CUDA_CHECK(cudaMalloc(&d_global_best_position, (size_t)NUM_TASKS * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_fitness, particle_bytes));
    CUDA_CHECK(cudaMalloc(&d_rng_states, rng_bytes));

    CUDA_CHECK(cudaMemcpy(d_task_times, task_times, task_bytes, cudaMemcpyHostToDevice));

    float *positions = (float *)malloc(position_bytes);
    float *velocities = (float *)malloc(position_bytes);
    float *personal_best_positions = (float *)malloc(position_bytes);
    float *fitness = (float *)malloc(particle_bytes);
    float *personal_best_fitness = (float *)malloc(particle_bytes);
    unsigned int *rng_states = (unsigned int *)malloc(rng_bytes);

    if (!positions || !velocities || !personal_best_positions ||
        !fitness || !personal_best_fitness || !rng_states) {
```

```
        fprintf(stderr, "Host memory allocation failed.\n");
        return EXIT_FAILURE;
    }

    // Initialize particles with random assignments and velocities.
    for (int i = 0; i < NUM_PARTICLES * NUM_TASKS; ++i) {
        positions[i] = (float)(rand() % NUM_PROCESSORS) + (float)(rand() % 1000) / 1000.0f;
        if (positions[i] >= NUM_PROCESSORS) positions[i] = NUM_PROCESSORS - 0.001f;
        velocities[i] = ((float)(rand() % 2001) / 1000.0f) - 1.0f;
        personal_best_positions[i] = positions[i];
        rng_states[i] = 123456789u + (unsigned int)(i * 747796405u + 2891336453u);
    }

    CUDA_CHECK(cudaMemcpy(d_positions, positions, position_bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_velocities, velocities, position_bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_personal_best_positions, personal_best_positions,
                          position_bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_rng_states, rng_states, rng_bytes, cudaMemcpyHostToDevice));

    int threads = 128;
    int blocks_particles = (NUM_PARTICLES + threads - 1) / threads;
    int blocks_elements = ((NUM_PARTICLES * NUM_TASKS) + threads - 1) / threads;

    // Evaluate initial population.
    evaluateFitness<<<blocks_particles, threads>>>(d_task_times, d_positions, d_fitness,
                                                   NUM_PARTICLES, NUM_TASKS);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaMemcpy(fitness, d_fitness, particle_bytes, cudaMemcpyDeviceToHost));

    int global_best_index = 0;
    float global_best_fitness = INFINITY;
    for (int p = 0; p < NUM_PARTICLES; ++p) {
        personal_best_fitness[p] = fitness[p];
        if (fitness[p] < global_best_fitness) {
            global_best_fitness = fitness[p];
            global_best_index = p;
        }
    }

    float *global_best_position = (float *)malloc((size_t)NUM_TASKS * sizeof(float));
    if (!global_best_position) {
        fprintf(stderr, "Global-best memory allocation failed.\n");
        return EXIT_FAILURE;
    }
    for (int t = 0; t < NUM_TASKS; ++t) {
        global_best_position[t] = positions[global_best_index * NUM_TASKS + t];
    }

    CUDA_CHECK(cudaMemcpy(d_global_best_position, global_best_position,
                          (size_t)NUM_TASKS * sizeof(float), cudaMemcpyHostToDevice));

    printf("\nInitial best makespan: %.0f\n", global_best_fitness);

    for (int iter = 1; iter <= MAX_ITERATIONS; ++iter) {
        updateParticles<<<blocks_elements, threads>>>(
            d_positions, d_velocities, d_personal_best_positions,
            d_global_best_position, d_rng_states,
            NUM_PARTICLES, NUM_TASKS, iter);
        CUDA_CHECK(cudaGetLastError());

        evaluateFitness<<<blocks_particles, threads>>>(
            d_task_times, d_positions, d_fitness, NUM_PARTICLES, NUM_TASKS);
        CUDA_CHECK(cudaGetLastError());
        CUDA_CHECK(cudaDeviceSynchronize());

        CUDA_CHECK(cudaMemcpy(fitness, d_fitness, particle_bytes, cudaMemcpyDeviceToHost));
        CUDA_CHECK(cudaMemcpy(positions, d_positions, position_bytes, cudaMemcpyDeviceToHost));

        int improved = 0;
        for (int p = 0; p < NUM_PARTICLES; ++p) {
            if (fitness[p] < personal_best_fitness[p]) {
                personal_best_fitness[p] = fitness[p];
                for (int t = 0; t < NUM_TASKS; ++t) {
                    personal_best_positions[p * NUM_TASKS + t] =
                        positions[p * NUM_TASKS + t];
                }
                improved = 1;
            }

            if (personal_best_fitness[p] < global_best_fitness) {
                global_best_fitness = personal_best_fitness[p];
```

```
                global_best_index = p;
                for (int t = 0; t < NUM_TASKS; ++t) {
                    global_best_position[t] =
                        personal_best_positions[p * NUM_TASKS + t];
                }
            }
        }

        // Transfer the updated personal and global best positions for the next GPU update.
        CUDA_CHECK(cudaMemcpy(d_personal_best_positions, personal_best_positions,
                              position_bytes, cudaMemcpyHostToDevice));
        CUDA_CHECK(cudaMemcpy(d_global_best_position, global_best_position,
                              (size_t)NUM_TASKS * sizeof(float), cudaMemcpyHostToDevice));

        if (iter == 1 || iter % 25 == 0 || iter == MAX_ITERATIONS) {
            printf("Iteration %3d | Best makespan: %.0f%s\n",
                   iter, global_best_fitness, improved ? "" : " (no personal-best update)");
        }
    }

    // Decode final global-best allocation and calculate processor loads.
    int processor_loads[NUM_PROCESSORS] = {0};
    int processor_task_counts[NUM_PROCESSORS] = {0};
    int assignments[NUM_TASKS];

    for (int t = 0; t < NUM_TASKS; ++t) {
        int processor = (int)floorf(global_best_position[t]);
        if (processor < 0) processor = 0;
        if (processor >= NUM_PROCESSORS) processor = NUM_PROCESSORS - 1;

        assignments[t] = processor;
        processor_loads[processor] += task_times[t];
        processor_task_counts[processor]++;
    }

    int final_makespan = 0;
    for (int p = 0; p < NUM_PROCESSORS; ++p) {
        if (processor_loads[p] > final_makespan) final_makespan = processor_loads[p];
    }

    printf("\n=== Final PSO Result ===\n");
    printf("Best makespan: %d\n", final_makespan);
    printf("Total task execution time: ");
    long long total_time = 0;
    for (int t = 0; t < NUM_TASKS; ++t) total_time += task_times[t];
    printf("%lld\n", total_time);
    printf("Ideal lower bound (ceil(total time / processors)): %lld\n",
           (total_time + NUM_PROCESSORS - 1) / NUM_PROCESSORS);

    printf("\nProcessor summary:\n");
    for (int p = 0; p < NUM_PROCESSORS; ++p) {
        printf("P%d: load = %4d | tasks = %3d\n",
               p, processor_loads[p], processor_task_counts[p]);
    }

    printf("\nTask assignments:\n");
    for (int t = 0; t < NUM_TASKS; ++t) {
        printf("T%-3d (time %3d) -> P%d\n", t, task_times[t], assignments[t]);
    }

    free(positions);
    free(velocities);
    free(personal_best_positions);
    free(fitness);
    free(personal_best_fitness);
    free(rng_states);
    free(global_best_position);

    cudaFree(d_task_times);
    cudaFree(d_positions);
    cudaFree(d_velocities);
    cudaFree(d_personal_best_positions);
    cudaFree(d_global_best_position);
    cudaFree(d_fitness);
    cudaFree(d_rng_states);

    return 0;
}
```

PSO Output

Compile and run the program using:

```
nvcc -O2 pso_task_allocation.cu -o pso
./pso
```

The program prints all task execution times, the best makespan at selected iterations, final processor loads, and each task's assignment. The following is the output format; numerical values are produced by the actual CUDA run.

```
=== Task Allocation using CUDA PSO ===
Tasks: 200 | Processors: 8 | Particles: 512 | Iterations: 500

Task execution times:
T0  : ...  T1  : ...  ...  T199: ...

Initial best makespan: <value>
Iteration   1 | Best makespan: <value>
Iteration  25 | Best makespan: <value>
...
Iteration 500 | Best makespan: <value>

=== Final PSO Result ===
Best makespan: <value>
Total task execution time: <value>
Ideal lower bound (ceil(total time / processors)): <value>

Processor summary:
P0: load = <value> | tasks = <value>
...
P7: load = <value> | tasks = <value>

Task assignments:
T0 (time <value>) -> P<id>
...
T199 (time <value>) -> P<id>
```

Short Analysis

The makespan measures the load on the busiest processor. A smaller makespan indicates a more balanced task allocation. The lower bound

$$C_{\mathrm{LB}} = \left\lceil \frac{\sum_{i=0}^{199} t_i}{8} \right\rceil$$

is a useful reference, although the largest individual task time can also impose a lower bound. Thus, a stronger lower bound is

$$C_{\mathrm{LB}} = \max\left(\left\lceil \frac{\sum_{i=0}^{199} t_i}{8} \right\rceil, \max_i t_i \right).$$

GA explores candidate assignments through selection, crossover, and mutation. Elitism preserves strong chromosomes, so its best-so-far makespan cannot increase. PSO adjusts candidate assignments using personal and global best positions; its global best-so-far makespan likewise cannot increase. Both algorithms are stochastic search methods and do not guarantee a globally optimal solution. Their performance should be compared using the final makespan, convergence trend, and runtime under the same task data and hardware. Since the current programs

print different progress details, use the final reported makespan from each actual run for a fair comparison.

The fitness evaluation is parallelized across candidate solutions on the GPU. However, genetic operations and PSO best-solution bookkeeping are performed on the CPU in these implementations. Consequently, the total speedup depends not only on the CUDA kernels but also on host-device data transfers and CPU-side work.

Conclusion

CUDA was used to evaluate task allocations in parallel for GA and PSO. Both methods seek to minimize the maximum processor load. The final makespan and processor-load distribution indicate the quality of each allocation. Actual output values should be inserted from the program runs before submitting the report.